Zewail City of Science, Technology and Innovation
University of Science and Technology
School of Computational Sciences and Artificial Intelligence

CSAI 203 - Fall 2026

Introduction to Software Engineering

Smart health & fitness tracker

Team Number: #
Team Members:
Youssef abdelaal 202400563
Eyad Eslam Ibrahim 202401218
Eyad Al-Saeed Ahmed 202508209
Representative Contact info: s-youssef.abdelaal@zewailcity.edu.eg

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to define the functional and non-functional requirements of the Smart Health & Fitness Tracker application. It provides a clear and consistent description of the system's objectives, scope, features, and expected behavior. This document serves as a reference for the project's design, implementation, and testing phases, ensuring that all team members share a common understanding of the system requirements.

1.2 Scope

The Smart Health & Fitness Tracker is a wellness application designed to help users monitor their daily physical activities and nutritional intake, establish personalized goals, and track their progress over time. The system aims to support healthier lifestyle habits by providing users with an organized way to manage their fitness and nutrition information.

The application will provide the following main capabilities:

- User Account and Profile Management: Allow users to create accounts, log in and out, and manage their personal information.

- Workout Tracking: Enable users to record workouts, view their workout history, edit or delete workout records, and estimate calories burned.

- Nutrition and Calorie Tracking: Allow users to manually record meals and food items, specify

quantities and calorie values, and monitor daily calorie intake against their goals.

- Goal Setting and Progress Monitoring: Enable users to establish customizable goals and view daily summaries and progress charts over different periods.

- Workout Routine Sharing: Allow users to create workout routines, publish them for other users to view, and save copies of shared routines for personal use.

The system focuses on personal wellness and fitness tracking. It is not intended to provide medical diagnoses, replace professional medical advice, or deliver clinical treatment.

1.3 Definitions, Acronyms, and Abbreviations

The following terms and abbreviations are used throughout this document.

| *Term* | *Definition* |

|---|---|

| *SRS* | *Software Requirements Specification: a document describing the requirements of a software system.* |

| *UI* | *User Interface: the screens and interactive elements through which users interact with the application.* |

| *UML* | *Unified Modeling Language: a standard modeling language used to represent software systems and their behavior and structure.* |

| *Functional Requirement (FR)* | *A requirement specifying a function or behavior that the system must provide.* |

| *Non-Functional Requirement (NFR)* | *A requirement specifying a quality attribute, performance expectation, or constraint of the system.* |

| *Workout* | *A physical activity session recorded by a user.* |

| *Workout Routine* | *An organized collection of exercises intended to form a workout plan.* |

| *Calorie Intake* | *The estimated number of calories consumed through food and meals.* |

| *Calories Burned* | *An estimate of the energy expended during physical activity.* |

| *Goal* | *A target established by a user to support personal fitness or nutrition objectives.* |

| *Progress Chart* | *A graphical representation of recorded activity, nutritional intake, or progress toward a goal.* |

1.4 References

The following references provide guidance for preparing and structuring this Software

Requirements Specification (SRS):

1. ISO/IEC/IEEE, ISO/IEC/IEEE 29148:2018 — Systems and Software Engineering — Life Cycle Processes — Requirements Engineering. Available at: https://www.iso.org/standard/72089.html

2. Object Management Group (OMG), OMG Unified Modeling Language (OMG UML), Version 2.5.1, December 2017. Available at: https://www.omg.org/spec/UML/2.5.1/

3. Mohamed Sami Rakha, Introduction to Software Engineering (CSAI 203), Course Project — Fall 2026, course project instructions. These references support requirements engineering and software modeling. The course project instructions serve as the primary source for the required structure and deliverables.

1.5 Overview

This document is organized into four main sections. Section 1 introduces the purpose, scope, terminology, and references of the project. Section 2 provides an overall description of the system, including its context, major functions, users, operating environment, constraints, documentation, and assumptions. Section 3 specifies the detailed functional and non-functional requirements, use case models and descriptions, conceptual class model, and external interface requirements. Section 4 contains supplementary information, including the data dictionary and glossary.

2. Overall Description

2.1 Product Perspective

a standalone web application for tracking health and fitness data. It does not belong to a larger product system and does not replace an existing system. It runs locally on the user's device and stores its data there, so it works without an internet connection. The only exchange with the outside world is file based, users can export a workout as a file and import workouts shared by others, or find ready existing workouts to choose from.

2.2 Product Functions

The Smart Health & Fitness Tracker provides a set of core functions designed to help users manage their fitness activities, monitor their nutritional intake, and track their progress toward personal wellness goals. The main functions of the system are summarized as follows:

1. User Account and Profile Management: The system allows users to create accounts, log in and out securely, and manage their personal profile information to support a personalized

tracking experience.

2. Workout Tracking: The system enables users to record their physical activities, maintain a history of completed workouts, manage existing workout records, and view estimated calories burned during exercise.

3. Nutrition and Calorie Tracking: The system allows users to record meals and food items manually, specify their quantities and calorie values, and monitor their daily calorie consumption.

4. Personalized Goal Management: The system enables users to establish and customize fitness and nutrition goals according to their personal objectives.

5. Progress Monitoring and Visualization: The system provides daily summaries and graphical representations of workout activities, calorie intake, and progress toward established goals over selected periods.

6. Workout Routine Management and Sharing: The system allows users to create and organize workout routines, publish them for other users to view, and save copies of shared routines for personal use.

Together, these functions provide an integrated wellness tracking experience that helps users organize their fitness and nutrition records, understand their progress, and maintain awareness of their personal goals. The system is intended to support general wellness and fitness management rather than provide medical diagnosis or treatment.

2.3 User Classes and Characteristics

The system has one user class, a person who registers, logs in, and tracks their own meals, workouts, body measurements, and other daily data. All users have the same permissions, and there is no particular administrator role. A user's fitness level does not change what they can do in the system. No technical or fitness knowledge is required, because the interface guides the user through each task. The system also has an internal actor, the System, which performs scheduled actions such as sending reminders without user input.

2.4 Software Stakeholders

The system has mainly three groups of stakeholders. The first is the end users: anyone who wants to track their meals, workouts, or other daily data, and who is mainly interested in entering that data easily and reviewing their progress over time. The second is the development team, the three students who design, build, test, and

maintain the system. They need clear and consistent requirements to guide their work. The third are gym coaches / gyms in general, they use the app to create workouts for trainers and track their progress

2.5 Operating Environment

“Assuming what we use in the lab for now, since nothing is announced, “agile” SMH 😒”

The system runs as a web application, so any device with a modern web browser can use it. The server side is written in Python using the Flask framework `TBD` and runs locally, with no external server or internet connection required. Data is stored in a local database `TBD`. There are no special hardware requirements beyond a device that can run a browser.

2.6 Design and Implementation Constraints

- **Technology:** the implementation uses Python and Flask `TBD`, since these are the tools taught in the course.
- **Offline operation:** the system works without an internet connection, so it does not depend on external APIs or online services.
- **Units:** users can choose their preferred unit system (for example kg or lb, cm or in) at any time to include malleability, and the system converts values correctly.
- **Calculation accuracy:** health calculations such as BMR, TDEE, and calories burned use the most accurate formulas available `[e.g., Mifflin-St Jeor for BMR]`.
- **Disclaimers:** the system must show clear disclaimers stating that its results are estimates for general information and are not medical advice, avoiding legal issues.

2.7 User Documentation

The system will come with the following help material:

- A FAQ that answers common questions about using the system, guiding them with any problem they might need.
- In-app help text and tooltips on the main screens, to ease the use for any beginner.
- The README file in the GitHub repository covering installation and how to run / use the application.
- Tutorials that walk a new user through the main tasks, such as logging a workout and setting a goal, in addition to motivating them to reach their goal.

2.8 Assumptions and Dependencies

Assumptions:

- The system calculates results from the data the user enters. It does not check whether the data is realistic, but it checks the validity for the info `[e.g., user can not use letters in the age field]`. ; the user is responsible for entering correct values.
- Users have a device with a modern web browser, and the machine running the application has Python installed.
- The device clock is correct, because goals, streaks, and reminders depend on dates and times.
- Workouts are shared by exchanging exported files, not through an online service.

3. Specific Requirements

3.1 Functional Requirements

The following functional requirements define the expected behavior of the Smart Health & Fitness Tracker. Each requirement specifies a unique system function, its detailed behavior, relevant alternative scenarios or error conditions, and the expected outcome. These requirements will serve as a reference for the system's design, implementation, and testing.

A. User Account and Profile Management

FR-01 — User Registration

Description:

The system shall allow a new user to create an account by providing the required registration information, including a name, email address, and password. The system shall validate the submitted information, ensure that the email address is not already associated with an existing account, and enforce the application's password requirements. Once the information is validated, the system shall create the account and securely store the user's credentials. The system shall display appropriate feedback to confirm successful registration.

Alternative Scenarios and Error Conditions:

- If one or more required fields are missing, the system shall identify the missing information and request correction.

- If the email address is already registered, the system shall reject the registration and inform the user.

- If the email address or password does not satisfy the validation rules, the system shall display an appropriate error message.

- If account creation fails because of a system or database error, the system shall notify the user without creating an incomplete account.

Expected Outcome: A valid new user account is created, and the user receives confirmation of successful registration.

FR-02 — User Login and Logout

Description:

The system shall allow registered users to log in using their registered email address and password. It shall verify the supplied credentials and establish an authenticated session when they are valid. After successful authentication, the system shall provide access to the user's personal dashboard and authorized features. The system shall also provide a logout function that terminates the active session and prevents further access to protected pages until the user logs in again.

Alternative Scenarios and Error Conditions:

- If the user enters an incorrect email address or password, the system shall deny access and

display a generic authentication error.

- If required login fields are empty, the system shall request the missing information.

- If the authentication service is temporarily unavailable, the system shall display an appropriate error message.

- If an unauthenticated user attempts to access a protected feature, the system shall redirect the user to the login page.

Expected Outcome: Authorized users can access their accounts, while unauthenticated users cannot access protected information or features.

FR-03 — View and Update Personal Profile

Description:

The system shall allow authenticated users to view and update their personal profile information, including their name, date of birth, height, weight, and activity level, where applicable. The system shall validate changes before saving them and use relevant profile information to support personalized fitness and nutrition goals. Users shall be able to review their saved information and modify it whenever necessary.

Alternative Scenarios and Error Conditions:

- If a required field is empty, the system shall request valid information before saving the changes.

- If a numerical value, such as height or weight, falls outside the application's accepted range, the system shall reject the value and explain the validation issue.

- If saving the profile fails, the system shall inform the user and preserve the previously saved information.

- If the user's session has expired, the system shall require authentication before allowing profile changes.

Expected Outcome: The user's profile contains valid, up-to-date personal information that can support personalized tracking.

B. Workout Tracking

FR-04 — Record a Workout

Description:

The system shall allow authenticated users to record a completed workout by entering the activity or exercise name, workout date, duration, and any additional supported details, such as intensity, repetitions, or notes. The system shall validate the required fields and associate each saved workout with the user who created it. Users shall be able to record different types of

physical activities supported by the application.

Alternative Scenarios and Error Conditions:

- If the activity name, date, or duration is missing, the system shall request the missing information.

- If the user enters a negative duration or an invalid date, the system shall reject the entry.

- If the workout cannot be saved because of a system error, the system shall display an error message and avoid creating a partial record.

- If the user is not authenticated, the system shall require login before saving a workout.

Expected Outcome: A valid workout record is stored and associated with the correct user account.

FR-05 — View Workout History

Description:

The system shall allow users to view a history of their previously recorded workouts. Each entry shall display relevant information, including the activity name, workout date, duration, and estimated calories burned when available. The system shall organize workout records chronologically and allow users to review their activity over time. The history shall contain only records belonging to the authenticated user.

Alternative Scenarios and Error Conditions:

- If the user has not recorded any workouts, the system shall display an informative empty-state message.

- If workout records cannot be retrieved, the system shall notify the user that the information is temporarily unavailable.

- If a user attempts to view another user's private workout history, the system shall deny access.

Expected Outcome: The user can review their own workout history and understand their recorded physical activity over time.

FR-06 — Edit or Delete a Workout

Description:

The system shall allow users to modify or delete workout records they own. When editing a workout, the system shall display the existing details, allow the user to make changes, validate the updated information, and save the modifications. When deleting a workout, the system shall request confirmation before permanently removing the record. Any summaries or progress calculations that depend on the modified or deleted workout shall reflect the updated data.

Alternative Scenarios and Error Conditions:

- If the updated workout information is invalid, the system shall reject the changes and explain the validation error.

- If the user cancels the deletion confirmation, the workout shall remain unchanged.

- If the specified workout does not exist or does not belong to the user, the system shall reject the operation.

- If an update or deletion fails, the system shall notify the user and preserve data integrity.

Expected Outcome: The user's workout records are updated or removed as requested, and related summaries remain consistent with the saved records.

FR-07 — Estimate Calories Burned During Workouts

Description:

The system shall estimate the calories burned during a workout using a defined calculation method and the available activity information. Depending on the selected method, the calculation may use the workout type, duration, intensity, and relevant user profile data. The system shall display the resulting estimate and associate it with the corresponding workout record. The estimate shall be clearly identified as an approximation rather than an exact measurement.

Alternative Scenarios and Error Conditions:

- If required calculation inputs are missing, the system shall request the necessary information or indicate that an estimate cannot be generated.

- If the selected activity is unsupported by the calculation method, the system shall inform the user that no estimate is available.

- If the calculation fails, the system shall preserve the workout record and display an appropriate message.

Expected Outcome: The user can view an estimated calorie expenditure for supported workouts without the system presenting the estimate as an exact medical measurement.

C. Nutrition and Calorie Tracking

FR-08 — Record Meals and Food Items

Description:

The system shall allow authenticated users to record meals by manually entering meal information, including the meal date, meal type, food item names, quantities, serving sizes, and calorie values. A meal may contain multiple food items, and each item shall have its own quantity and calorie information. The system shall validate the entered values and associate

each meal with the user's account. Users shall be able to record meals consumed at different times throughout the day.

Alternative Scenarios and Error Conditions:

- If required meal information is missing, the system shall request the missing fields.

- If a quantity or calorie value is negative or otherwise invalid, the system shall reject the entry.

- If a meal contains no valid food items, the system shall prevent it from being saved as a completed meal.

- If the system fails to save the meal, it shall notify the user and avoid creating an incomplete record.

Expected Outcome: A valid meal and its associated food items are saved under the user's account and become available for daily calorie tracking.

FR-09 — View and Manage Meal History

Description:

The system shall allow users to view previously recorded meals and their associated food items, quantities, and calorie values. The system shall organize meal records by date and display the information needed to review daily nutritional intake. Users shall also be able to edit meal details or remove a meal or individual food item from their records. Following any modification, the system shall update the affected nutritional totals.

Alternative Scenarios and Error Conditions:

- If no meals have been recorded for the selected date, the system shall display an informative message.

- If the user enters invalid information while editing a meal, the system shall reject the changes and identify the invalid fields.

- If the requested meal or food item does not exist or does not belong to the user, the system shall reject the operation.

- If an update or deletion fails, the system shall notify the user and retain the last successfully saved data.

Expected Outcome: The user can review and manage their meal history, and the saved records remain consistent with the nutritional summaries.

FR-10 — Calculate Daily Calorie Intake

Description:

The system shall calculate the user's total daily calorie intake by summing the calorie values of all valid food items recorded for the selected date. The calculation shall account for the quantity

or serving size entered for each food item. When a user adds, edits, or deletes a meal or food item, the system shall recalculate the daily total using the updated records. The calculated total shall be displayed in the nutrition tracking interface.

Alternative Scenarios and Error Conditions:

- If no food items have been recorded for the selected date, the system shall display a total of zero calories or an equivalent empty-state indication.

- If a food item has missing or invalid calorie information, the system shall exclude it from the calculation until valid information is provided and notify the user where appropriate.

- If the system cannot retrieve the required records, it shall indicate that the total is temporarily unavailable rather than display an incorrect value.

Expected Outcome: The user receives an up-to-date daily calorie intake total based on the saved food records.

FR-11 — Set and Monitor a Daily Calorie Target

Description:

The system shall allow users to establish and modify a personalized daily calorie intake target. The target shall be stored with the user's account and used to compare the daily calorie intake against the intended limit or goal. The system shall display the current intake, the target, and the remaining calories when the intake is below the target. If the intake exceeds the target, the system shall display the amount by which the target has been exceeded. The system shall recalculate these values whenever the meal records or target change.

Alternative Scenarios and Error Conditions:

- If the user enters a missing, zero, negative, or otherwise invalid target, the system shall reject the value and request a valid target.

- If the user has not configured a target, the system shall prompt them to set one and shall not assume an arbitrary personalized target.

- If the daily intake exceeds the target, the system shall display the updated total and indicate the excess without preventing further meal entries.

Expected Outcome: The user can monitor daily calorie intake relative to a personalized target and understand whether the target has been met or exceeded.

D. Goal Setting and Progress Monitoring

FR-12 — Create and Customize Fitness Goals

Description:

The system shall allow authenticated users to create personalized fitness goals based on

supported measurements, such as workout frequency, workout duration, or other fitness metrics tracked by the application. Each goal shall include a goal type, a target value, and any applicable start date or target date. The system shall validate the goal information and associate the goal with its owner. Users shall be able to establish multiple goals, provided each goal uses a supported measurement.

Alternative Scenarios and Error Conditions:

- If required goal information is missing, the system shall request completion of the relevant fields.

- If the target value is invalid for the selected goal type, the system shall reject the goal and explain the problem.

- If the selected goal type is unsupported, the system shall inform the user of the available goal types.

- If the goal cannot be saved, the system shall display an error and avoid creating an incomplete goal.

Expected Outcome: A valid, personalized fitness goal is created and made available for progress tracking.

FR-13 — Update and Manage Goals

Description:

The system shall allow users to view their existing goals, modify target values or applicable dates, and mark goals as completed or discontinue goals that are no longer relevant. The system shall validate any changes and update the goal's status accordingly. Changes to a goal shall affect subsequent progress calculations and visualizations. The system shall preserve the user's recorded workout and meal history when a goal is modified or discontinued.

Alternative Scenarios and Error Conditions:

- If a user submits invalid changes, the system shall reject them and preserve the previously saved goal.

- If the requested goal does not exist or does not belong to the user, the system shall reject the operation.

- If a goal has already been completed or discontinued, the system shall prevent inconsistent status changes and communicate the permitted actions.

- If saving a change fails, the system shall notify the user without falsely confirming success.

Expected Outcome: Users can maintain accurate, up-to-date goals while retaining the historical records used to evaluate their progress.

FR-14 — View Progress Charts

Description:

The system shall provide graphical representations of the user's recorded fitness and nutrition information over selectable periods, such as daily or weekly views. Supported visualizations shall include workout activity, recorded workout duration, daily calorie intake, and progress toward applicable goals. The system shall derive the displayed information from saved records and update the charts when relevant data changes. Each chart shall clearly identify the metric and time period being displayed.

Alternative Scenarios and Error Conditions:

- If insufficient data exists for the selected period, the system shall display the available data and indicate that there is insufficient information for a complete trend.

- If the selected date range is invalid, the system shall request a valid range.

- If progress data cannot be retrieved or processed, the system shall notify the user instead of presenting misleading results.

Expected Outcome: The user can visualize fitness and nutrition trends and evaluate progress over the selected period using available records.

FR-15 — View Daily Health and Fitness Summary

Description:

The system shall provide a daily summary combining the user's relevant workout and nutrition information. The summary shall display the number or duration of recorded workouts, estimated calories burned where available, total calorie intake, progress against the daily calorie target, and the status of relevant fitness goals. The summary shall use records associated with the selected date and authenticated user. It shall refresh its displayed values when relevant records are added, modified, or deleted.

Alternative Scenarios and Error Conditions:

- If the user has not recorded workouts or meals for the selected date, the system shall display appropriate empty-state information without inventing activity.

- If one category of data is unavailable, the system shall display the available summary information and identify the missing category where appropriate.

- If the system fails to retrieve the summary data, it shall inform the user that the summary cannot currently be generated.

Expected Outcome: The user receives a consolidated overview of daily fitness and nutrition activity and can quickly assess progress toward personal goals.

E. Workout Routine Management and Sharing

FR-16 — Create a Workout Routine

Description:

The system shall allow authenticated users to create a workout routine by providing a title, an optional description, and an ordered collection of exercises. For each exercise, users shall be able to specify supported details, such as the number of sets, repetitions, or workout duration, where applicable. The system shall validate the routine information and ensure that the routine contains at least one valid exercise before it can be saved. Each routine shall be associated with its creator and stored for future editing or sharing.

Alternative Scenarios and Error Conditions:

- If the routine title is missing or invalid, the system shall request a valid title.

- If the user attempts to save a routine without any exercises, the system shall request at least one valid exercise.

- If an exercise contains invalid details, such as a negative number of repetitions, the system shall identify the invalid information and request correction.

- If saving fails, the system shall notify the user and avoid creating an incomplete routine.

Expected Outcome: A valid workout routine is saved under the creator's account and is available for further management.

FR-17 — Edit or Delete a Workout Routine

Description:

The system shall allow users to edit or delete workout routines they have created. Editing shall allow the owner to update the routine's title, description, exercise list, exercise order, and supported exercise details. The system shall validate all changes before saving them. When deleting a routine, the system shall request confirmation before removing it. The system shall ensure that only the routine's owner can modify or delete it and that changes do not unintentionally alter independent copies saved by other users.

Alternative Scenarios and Error Conditions:

- If the user submits invalid routine information, the system shall reject the changes and identify the fields requiring correction.

- If the user cancels a deletion request, the routine shall remain unchanged.

- If the routine does not exist or the user is not its owner, the system shall reject the operation.

- If the routine is being accessed by another user while its owner modifies or deletes it, the system shall handle the change consistently and avoid displaying stale or inaccessible

information as though it were current.

Expected Outcome: The routine owner can maintain or remove their own routines, while other users' independently saved copies remain unaffected by changes to the original routine.

FR-18 — Publish and Unpublish Workout Routines

Description:

The system shall allow routine owners to publish their saved workout routines so that other users can discover and view them. Before publication, the system shall verify that the routine contains the required information and at least one valid exercise. The system shall make published routines available through the shared routine browsing feature. Owners shall also be able to unpublish their routines, after which the routines shall no longer appear as publicly available items. Publication status shall be maintained separately from the user's ownership of the routine.

Alternative Scenarios and Error Conditions:

- If a routine does not satisfy the publication requirements, the system shall prevent publication and explain what must be corrected.

- If an unauthenticated user attempts to publish a routine, the system shall require login.

- If a user attempts to publish or unpublish a routine they do not own, the system shall deny the operation.

- If publication fails, the system shall preserve the routine and its previous publication status.

Expected Outcome: Users can control whether their routines are publicly available, and only eligible routines are displayed in the shared routine collection.

FR-19 — Browse and View Shared Workout Routines

Description:

The system shall provide a shared routine collection where users can browse published workout routines created by other users. The system shall display relevant information for each routine, including its title, description, creator, and exercise list. Users shall be able to select a routine to view its details before deciding whether to save a copy. Only routines that are currently published and accessible to the requesting user shall be displayed as available shared routines.

Alternative Scenarios and Error Conditions:

- If no published routines are available, the system shall display an informative message.

- If a routine has been unpublished or removed since the user opened the collection, the system shall no longer present it as available for viewing or copying.

- If the requested routine cannot be found, the system shall inform the user that it is unavailable.

- If shared routine data cannot be retrieved, the system shall display an appropriate error message and allow the user to try again.

Expected Outcome: Users can discover and inspect available workout routines before deciding whether to save one for personal use.

FR-20 — Save a Copy of a Shared Workout Routine

Description:

The system shall allow authenticated users to save a copy of a published workout routine created by another user. The saved copy shall be associated with the receiving user's account and shall contain the routine's relevant information, including its title, description, and exercise details. The system shall maintain the distinction between the original routine and the saved copy so that users can manage their own copies without modifying the original creator's routine. The saved copy shall be available in the user's personal routine collection for future reference and use.

Alternative Scenarios and Error Conditions:

- If the user is not authenticated, the system shall request login before saving a copy.

- If the original routine is no longer published or accessible, the system shall prevent the user from saving a new copy and display an appropriate message.

- If the user attempts to save the same routine repeatedly, the system shall follow a consistent policy, such as preventing duplicate copies or informing the user that a copy already exists.

- If copying fails, the system shall notify the user and ensure that no incomplete copy is added to the personal collection.

Expected Outcome: A valid independent copy of the selected shared routine is saved to the user's personal collection without granting permission to modify the original routine.

End of Section 3.1 — Functional Requirements

These 20 functional requirements cover the five agreed areas of the Smart Health & Fitness Tracker: account and profile management, workout tracking, nutrition and calorie tracking, goal setting and progress monitoring, and workout routine sharing.

3.2 Use Case Model

3.2.1 Use Case Diagram

Figures 1–3 present the system’s major use cases. The User is the primary actor.

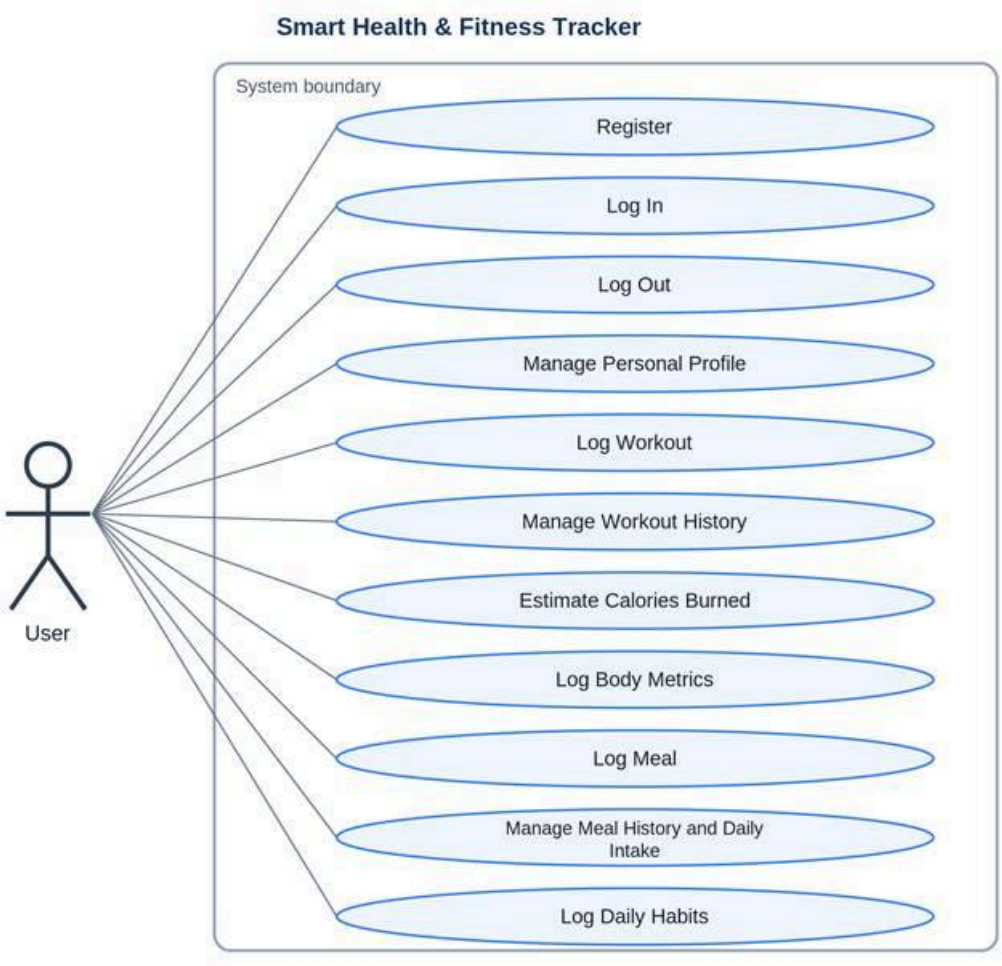


Figure 1. Use Case Diagram

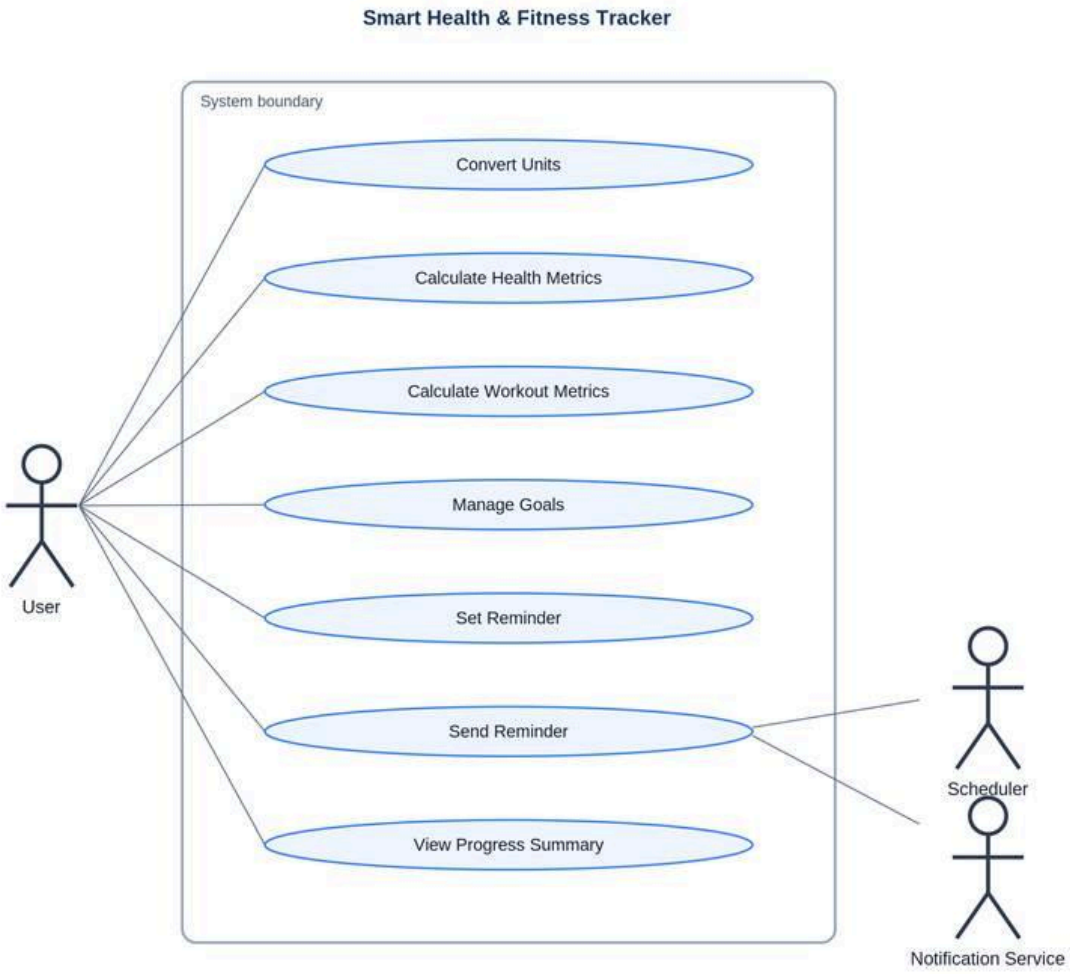
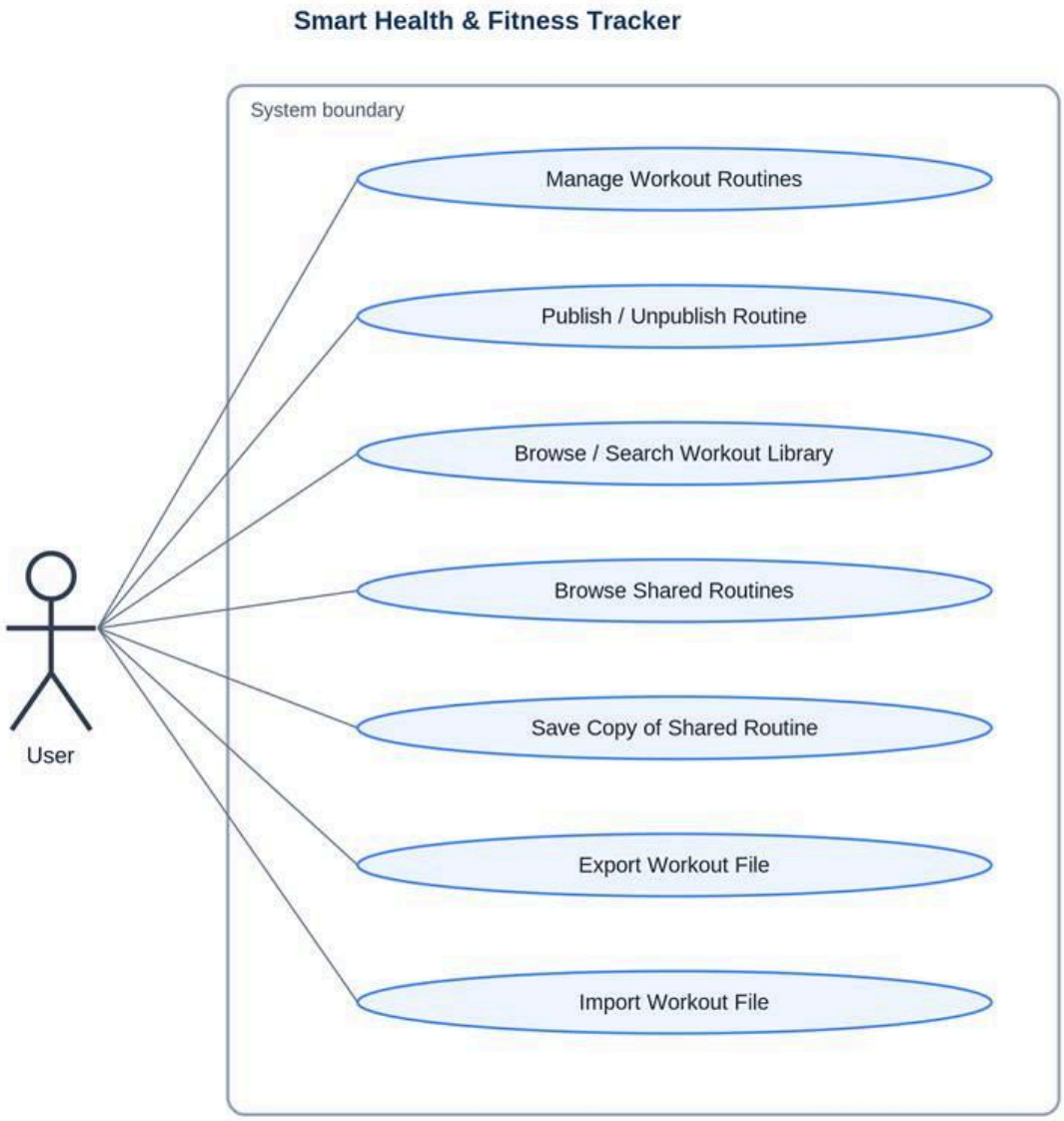


Figure 2. Use Case Diagram



3.2.2 Use Case Descriptions

Use Case	***Short Description***
UC-01 — Register	*The user creates an account by entering the required details.*
UC-02 — Log In	*The user enters their credentials to open the dashboard.*
UC-03 — Log Out	*The user ends the current session.*
UC-04 — Manage Profile	*The user views or updates personal information.*
UC-05 — Log Workout	*The user records a workout and its basic details.*

UC-06 — Manage Workout History	*The user views, edits, or deletes saved workouts.*
UC-07 — Estimate Calories Burned	*The system estimates calories burned during a workout when enough information is available.*
UC-08 — Log Body Metrics	*The user saves body measurements such as weight.*
UC-09 — Log Meal	*The user records a meal and its food items.*
UC-10 — Manage Meal History	*The user reviews or edits meals and checks daily calorie intake.*
UC-11 — Log Daily Habits	*The user records habits such as water intake and sleep.*
UC-12 — Convert Units	*The system converts measurements between supported units.*
UC-13 — Calculate Health Metrics	*The system calculates health metrics from the user's available information.*
UC-14 — Calculate Workout Metrics	*The system calculates supported workout results from entered details.*
UC-15 — Manage Goals	*The user creates, updates, or completes personal fitness goals.*
UC-16 — Set Reminder	*The user creates a reminder and chooses when it should appear.*
UC-17 — Send Reminder	*The system displays reminders at their scheduled times.*
UC-18 — View Progress	*The user views a summary of workout, meal, and goal progress.*
UC-19 — Manage Workout Routine	*The user creates, edits, or deletes a workout routine.*
UC-20 — Publish Routine	*The user changes whether a routine is available for sharing.*
UC-21 — Search Workout Library	*The user searches available workout routines.*
UC-22 — Browse Shared Routines	*The user views workout routines shared by others.*
UC-23 — Save Shared Routine	*The user saves a copy of an available shared routine.*
UC-24 — Export Workout File	*The user exports a workout routine to a file.*
UC-25 — Import Workout File	*The user imports a supported workout file into the system.*

3.3 Domain Model

3.3.1 Conceptual Class Diagram

Figure 1: Account and data logging

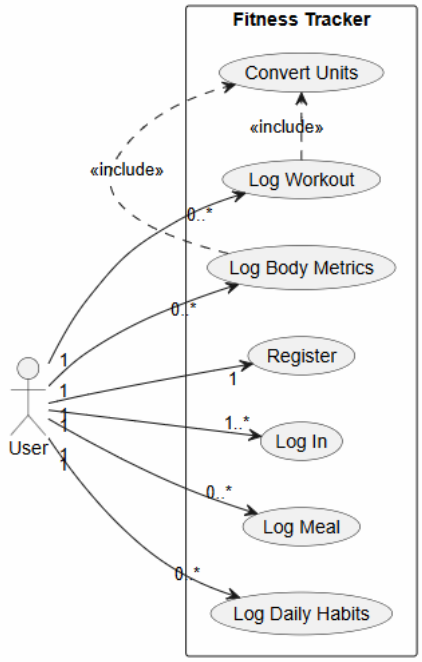


Figure 2: Calculators, goals and reminders

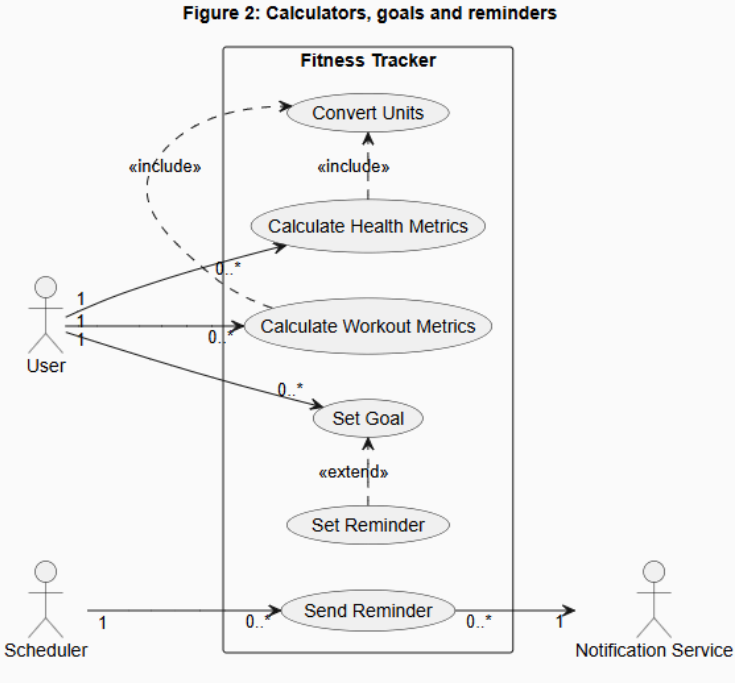
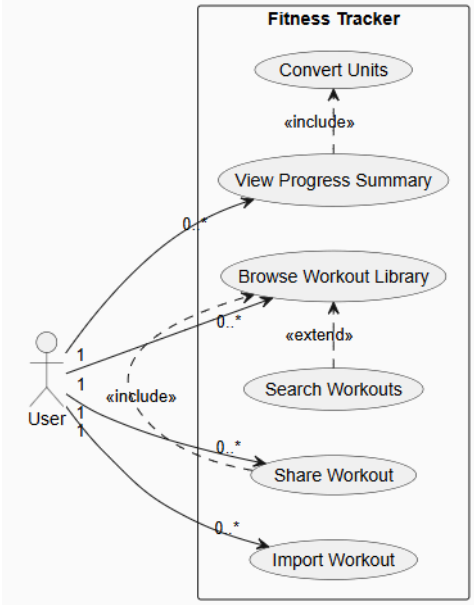


Figure 3: Progress summary and workout library



3.3.2 Class Descriptions

UC_Figure1:-

Register

- **Actor:** User
- **Description:** Creates a local account.
- **Trigger:** The User selects “Register”.
- **Preconditions:** The app is running and no account exists.
- **Postconditions:** The account is saved and the User can log in.
- **Normal flow:**
    1. The User enters a username, password, and profile details (age, sex, height).
    2. The system shows the health disclaimer and the User accepts.
    3. The system saves the account (password hashed) and shows Log In.
- **Alternative flow:** A1. The User cancels and nothing is saved.
- **Exceptions:** E1. A required field is empty. E2. The username is already taken.
- **Includes:** None
- **Priority / frequency:** High / once

Log In

- **Actor:** User
- **Description:** Signs in to reach their data.
- **Trigger:** The User opens the app or selects “Log In”.
- **Preconditions:** An account exists.
- **Postconditions:** A session is started and the dashboard is shown.
- **Normal flow:**
    1. The User enters a username and password.
    2. The system checks them against the local database.
    3. The system shows the dashboard.
- **Alternative flow:** A1. The User selects “Register” and Register starts.
- **Exceptions:** E1. The credentials are wrong or empty, so an error is shown.
- **Includes:** None
- **Priority / frequency:** High / every session

Log Workout

- **Actor:** User
- **Description:** Records a workout with its exercises, sets, reps, weight, and duration.

- **Trigger:** The User selects “Log Workout”.
- **Preconditions:** The User is logged in.
- **Postconditions:** The workout is saved with its date.
- **Normal flow:**
    1. The User enters the date and the exercises, choosing a unit for each weight.
    2. The system converts the weights to the stored unit (Convert Units).
    3. The User submits and the system saves and confirms.
- **Alternative flows:** A1. The User adds more exercises before saving. A2. The User cancels and nothing is saved.
- **Exceptions:** E1. A required field is empty or not a number. E2. The save fails and the entries are kept.
- **Includes:** Convert Units
- **Priority / frequency:** High / several times a week

Log Body Metrics

- **Actor:** User
- **Description:** Records body weight and measurements.
- **Trigger:** The User selects “Log Body Metrics”.
- **Preconditions:** The User is logged in.
- **Postconditions:** The entry is saved with its date.
- **Normal flow:**
    1. The User enters the date, weight, measurements, and unit.
    2. The system converts the values to the stored unit (Convert Units).
    3. The system saves and confirms.
- **Alternative flow:** A1. The User fills in only some of the fields.
- **Exceptions:** E1. All fields are empty or not numbers.
- **Includes:** Convert Units
- **Priority / frequency:** High / weekly

Log Meal

- **Actor:** User
- **Description:** Records a meal with its calories and optional macros.
- **Trigger:** The User selects “Log Meal”.
- **Preconditions:** The User is logged in.
- **Postconditions:** The meal is saved and the day’s total is updated.

- **Normal flow:**
    1. The User enters the meal name, type, and calories, plus protein, carbs, and fat if wanted.
    2. The system saves the meal and updates the day's total.
- **Alternative flow:** A1. The User adds another meal.
- **Exceptions:** E1. The name or calories are missing, or calories are not a number.
- **Includes:** None
- **Priority / frequency:** High / daily

Log Daily Habits

- **Actor:** User
- **Description:** Records water intake, sleep, and mood or energy for a day.
- **Trigger:** The User selects "Log Daily Habits".
- **Preconditions:** The User is logged in.
- **Postconditions:** The entries are saved for that date.
- **Normal flow:**
    1. The User enters the date, water intake, bedtime and wake time, and a mood rating.
    2. The system saves and confirms.
- **Alternative flow:** A1. The User fills in only some of the habits.
- **Exceptions:** E1. Nothing is filled in, or a value is not a number.
- **Includes:** None
- **Priority / frequency:** Medium / daily

Convert Units

- **Actor:** User, indirectly, through the including use case
- **Description:** Converts a value between metric and imperial units.
- **Trigger:** An including use case passes a value and its unit.
- **Preconditions:** The value is a number and its unit is known.
- **Postconditions:** The value is returned in the required unit.
- **Normal flow:**
    1. The system receives the value and unit.
    2. The system applies the conversion factor.
    3. The system returns the result.
- **Alternative flow:** A1. The unit already matches, so the value is returned unchanged.
- **Exceptions:** E1. The unit is not supported, so an error is returned to the calling use case.
- **Included by:** Log Workout, Log Body Metrics, Calculate Health Metrics, Calculate Workout

Metrics, View Progress Summary

- **Priority / frequency:** High / every entry

UC_Figure2:-

Calculate Health Metrics

- **Actor:** User
- **Description:** Calculates BMI, BMR, TDEE, and a daily calorie target.
- **Trigger:** The User selects “Health Calculator”.
- **Preconditions:** The User is logged in.
- **Postconditions:** The results are shown with the health disclaimer.
- **Normal flow:**
    1. The User enters or confirms weight, height, age, sex, activity level, and unit.
    2. The system converts the values (Convert Units).
    3. The system calculates the results and shows them.
- **Alternative flow:** A1. The User runs the calculation again with different values.
- **Exceptions:** E1. A required value is missing or not a number.
- **Includes:** Convert Units
- **Priority / frequency:** High / occasional

Calculate Workout Metrics

- **Actor:** User
- **Description:** Estimates one-rep max, heart rate zones, or calories burned.
- **Trigger:** The User selects “Workout Calculator”.
- **Preconditions:** The User is logged in.
- **Postconditions:** The result is shown with the health disclaimer.
- **Normal flow:**
    1. The User picks a calculation and enters its values and unit.
    2. The system converts the values (Convert Units).
    3. The system calculates the result and shows it.
- **Alternative flow:** A1. The User picks another calculation.
- **Exceptions:** E1. A required value is missing or not a number.
- **Includes:** Convert Units

- **Priority / frequency:** Medium / occasional

Set Goal

- **Actor:** User
- **Description:** Sets a daily or weekly goal for steps, water, workouts, or sleep.
- **Trigger:** The User selects “Set Goal”.
- **Preconditions:** The User is logged in.
- **Postconditions:** The goal is saved and tracked in the progress summary.
- **Normal flow:**
    1. The User picks a goal type and enters the target value and period.
    2. The system saves the goal.
    3. The system confirms.
- **Alternative flows:** A1. The User adds a reminder (Set Reminder). A2. The User cancels and nothing is saved.
- **Exceptions:** E1. The target is missing or not a number.
- **Extended by:** Set Reminder
- **Priority / frequency:** High / occasional

Set Reminder

- **Actor:** User
- **Description:** Adds an optional reminder to a goal.
- **Trigger:** The User chooses “Add reminder” after saving a goal.
- **Preconditions:** A goal has just been created in Set Goal.
- **Postconditions:** The reminder is saved and scheduled.
- **Normal flow:**
    1. The User picks the reminder time and repeat (daily or weekly).
    2. The system saves the reminder linked to the goal.
- **Alternative flow:** A1. The User skips it and the goal stays without a reminder.
- **Exceptions:** E1. The time is missing.
- **Extends:** Set Goal, after the goal is saved
- **Priority / frequency:** Low / once per goal

Send Reminder

- **Primary actor:** Scheduler
- **Secondary actor:** Notification Service

- **Description:** Shows a reminder to the User at its scheduled time.
- **Trigger:** A reminder's scheduled time is reached.
- **Preconditions:** A reminder exists and the app is open in the browser.
- **Postconditions:** The reminder is shown to the User.
- **Normal flow:**
    1. The Scheduler signals that a reminder time has arrived.
    2. The system builds the message from the goal.
    3. The Notification Service displays it.
- **Alternative flow:** A1. The User dismisses the reminder.
- **Exceptions:** E1. The browser blocks notifications, so the system shows the reminder inside the app instead.
- **Includes:** None
- **Priority / frequency:** Medium / daily

UC_Figure3:-

View Progress Summary

- **Actor:** User
- **Description:** Shows summaries, charts, personal records, goal progress, and streaks.
- **Trigger:** The User selects "Progress".
- **Preconditions:** The User is logged in.
- **Postconditions:** The summary is shown for the chosen period.
- **Normal flow:**
    1. The User picks the period (week or month) and unit.
    2. The system loads the entries and converts the values (Convert Units).
    3. The system shows the summaries, charts, records, goal progress, and streaks.
- **Alternative flow:** A1. The User changes the period or category.
- **Exceptions:** E1. There is no data for the period, so a "no data" message is shown.
- **Includes:** Convert Units
- **Priority / frequency:** High / daily

Browse Workout Library

- **Actor:** User

- **Description:** Lists the built-in and imported workouts.
- **Trigger:** The User selects “Workout Library”.
- **Preconditions:** The User is logged in.
- **Postconditions:** The workout list or a workout’s details are shown.
- **Normal flow:**
    1. The User opens the library.
    2. The system lists the workouts with name, muscle group, and level.
    3. The User opens one to see its exercises.
- **Alternative flow:** A1. The User searches the library (Search Workouts).
- **Exceptions:** E1. The library is empty, so a message is shown.
- **Extended by:** Search Workouts
- **Included by:** Share Workout
- **Priority / frequency:** Medium / weekly

Search Workouts

- **Actor:** User
- **Description:** Filters the library by keyword or category.
- **Trigger:** The User types a search term or picks a filter in the library.
- **Preconditions:** The library list is open (Browse Workout Library).
- **Postconditions:** The matching workouts are shown.
- **Normal flow:**
    1. The User types a keyword or picks a filter (muscle group or level).
    2. The system filters the list and shows the matches.
- **Alternative flow:** A1. The User clears the search and the full list returns.
- **Exceptions:** E1. Nothing matches, so a message is shown.
- **Extends:** Browse Workout Library, when the list is shown
- **Priority / frequency:** Medium / weekly

Share Workout

- **Actor:** User
- **Description:** Exports a workout as a file to give to someone else.
- **Trigger:** The User selects “Share” on a workout.
- **Preconditions:** The User is logged in.
- **Postconditions:** The workout file is saved to the User’s device.
- **Normal flow:**

1. The User selects a workout from the library (Browse Workout Library).
2. The User chooses “Share”.
3. The system exports the workout as a file and offers it for download.

- **Alternative flow:** A1. The User cancels.
- **Exceptions:** E1. The export fails, so an error is shown.
- **Includes:** Browse Workout Library
- **Priority / frequency:** Low / occasional

Import Workout

- **Actor:** User
- **Description:** Adds a workout from a shared file to the library.
- **Trigger:** The User selects “Import Workout”.
- **Preconditions:** The User is logged in and has a workout file.
- **Postconditions:** The workout is added to the library.
- **Normal flow:**
    1. The User selects a workout file.
    2. The system reads the file and checks its format.
    3. The system adds the workout to the library and confirms.
- **Alternative flow:** A1. The User cancels.
- **Exceptions:** E1. The file is unreadable or in the wrong format. E2. A workout with the same name exists.
- **Includes:** None
- **Priority / frequency:** Low / occasional.

3.4 Non-Functional Requirements

NFR-01: Performance

NFR: The system shall display the dashboard or daily fitness summary within 3 seconds for at least 95% of requests when tested with 100 concurrent users under normal operating conditions.

Test Method: Conduct a performance test using a testing tool to simulate 100 concurrent users accessing the dashboard and daily fitness summary. Record the response time for each request.

Success Criteria: At least 95% of requests must complete within 3 seconds, with no system

crashes during the test.

NFR-02: Security

NFR: The system shall prevent unauthorized users from accessing another user's private profile, workout records, meal records, and personal goals.

Test Method: Create two separate user accounts. Log in as the first user and attempt to access the second user's private data through the user interface and by directly requesting the relevant resources.

Success Criteria: All unauthorized access attempts must be denied, and no private data belonging to the second user may be displayed or modified.

NFR-03: Usability

NFR: A first-time user shall be able to record a workout and manually log a meal within 2 minutes per task, without external assistance, after receiving only basic instructions.

Test Method: Ask at least 10 participants who have not used the system before to complete both tasks. Measure the completion time and record whether assistance is required.

Success Criteria: At least 90% of participants must complete each task within 2 minutes without assistance or critical errors.

NFR-04: Reliability and Data Integrity

NFR: The system shall save valid workout and meal entries accurately and ensure that saved entries remain available after the user logs out and logs back in.

Test Method: Enter a predefined set of 50 valid workout and meal records. Log out, log back in, and compare the stored records and calculated calorie totals against the expected results.

Success Criteria: All 50 records must be retained with their entered values unchanged, and all calculated calorie totals must match the expected results.

NFR-05: Compatibility

NFR: The system shall provide its core features through the latest stable versions of Google Chrome, Mozilla Firefox, and Microsoft Edge on desktop devices, with no loss of functionality.

Test Method: Execute the same set of core functional tests—registration, login, workout logging, meal logging, goal management, progress viewing, and routine sharing—in each supported browser.

Success Criteria: All core functional tests must pass in all three browsers, and no browser-specific defect may prevent users from completing any core task.

3.5 External Interface Requirements

3.5.1 User Interface

The system has simple pages for registration, login, dashboard, profile, workouts, meals, body measurements, goals, reminders, and progress. Forms show clear labels and messages when information is missing or incorrect.

3.5.2 Hardware Interface

No special hardware is required. Users need a computer or device that can run a modern web browser. The computer running the application needs Python installed.

3.5.3 Software Interface

The system uses Python with the Flask framework and stores information in a local database. It works through a modern web browser and does not depend on external APIs or online services.

3.5.4 Communication Interface

The browser communicates with the locally running application using HTTP. Since the application is intended to run locally and offline, it does not require an internet connection.

4. Appendices

4.1 Appendix A: Data Dictionary

The database may contain the following main tables:

- *Users — account and login information.*
- *Profiles — personal details used for tracking.*
- *Workouts — workout date, type, duration, and related details.*
- *WorkoutRoutines — saved routines and exercises.*
- *Meals — meal records and food items.*
- *BodyMetrics — weight and other body measurements.*
- *DailyHabits — water intake, sleep, and other daily habits.*
- *Goals — user goals and their status.*
- *Reminders — reminder type, time, and status.*

4.2 Appendix B: Glossary

Term	***Meaning***
Dashboard	*The main page that summarizes the user's information.*
Workout	*A recorded physical activity or exercise session.*
Workout routine	*A planned group of exercises.*
Body metrics	*Measurements such as body weight.*

Calorie intake	*The estimated calories consumed from food.*
Calories burned	*The estimated energy used during activity.*
Goal	*A target the user wants to achieve.*
Reminder	*A message shown at a selected time.*
Local database	*A database stored on the computer running the application.*
Flask	*A Python framework used to build web applications.*
HTTP	*A protocol used by a browser to communicate with a web application.*